

Écrire des instructions efficaces que Claude lit à chaque session

Ce qu'il faut inclure

Le CLAUDE.md efficace contient trois blocs essentiels : la stack technique, les commandes de build/test, et les conventions critiques. Tout ce que Claude peut déduire du code lui-même (structure de dossiers, framework détecté, package manager) n'a pas besoin d'être documenté.

```
## Stack  Next.js 14, TypeScript, PostgreSQL/Prisma, pnpm
## Commandes  - `pnpm dev` - démarrer le serveur
- `pnpm test` - lancer les tests - `pnpm build`
- build production  ## Conventions critiques  -
Toujours pnpm, jamais npm ni yarn  -
Composants fonctionnels uniquement  -
Fichiers kebab-case (mon-composant.tsx)
```

```
└─ security.md (invariants de sécurité)
└─ api.md (conventions REST)
```

CLAUDE.md reste lisible d'un coup d'oeil. Les règles détaillées vivent dans `.claude/rules/`, chargées automatiquement à chaque session.

Ce qu'il ne faut PAS inclure

A éviter	Raison
Documentation narrative longue	Dilue les règles critiques
Contexte historique (« En 2023 on a décidé... »)	N'aide pas Claude à agir
Références à des fichiers entiers	Utiliser <code>@chemin/fichier</code> à la place
Règles que Claude respecte déjà	Bruit inutile, coût en tokens

Taille recommandée

Garder CLAUDE.md entre 4 et 8 KB au total (tous niveaux combinés). Les fichiers dépassant 16K tokens dégradent la cohérence du modèle. Boris Cherny (créateur de Claude Code) maintient un CLAUDE.md de 2,5K tokens pour son équipe, capturant les conventions sur plusieurs mois d'usage réel.

Règle pratique : si Claude fait la même erreur deux fois, ajouter la règle. Ne pas tout documenter préventivement.

Structure hiérarchique

```
~/ .claude/CLAUDE.md (global, tous projets)
/projet/CLAUDE.md (projet, comité)
/projet/.claude/CLAUDE.md (local, gitignore)
```

Dans un monorepo, Claude charge le CLAUDE.md racine puis fusionne le CLAUDE.md du sous-dossier actif. Les règles enfants complètent les règles parentes sans les effacer.

Import explicite avec @

```
# Dans CLAUDE.md @docs/conventions/coding-standards.md
@docs/conventions/architecture.md
```

Les fichiers importés avec `@chemin` se chargent à la demande, uniquement quand ils sont référencés. Ils ne consomment des tokens qu'au moment de leur utilisation, contrairement aux fichiers dans `.claude/rules/` qui chargent systématiquement au démarrage.

Architecture modulaire recommandée

```
.claude/ └─ CLAUDE.md (index, < 100 lignes)
└─ rules/ └─ testing.md (seuils de couverture)
```